TravelMemory

MERN Stack — AWS EC2 Deployment

Frontend	Backend API	Root Domain
https://www.draxil.site	https://api.draxil.site	https://draxil.site

Student Name	HeroVired ID	Submission Date
Kaushal Bhupendra Patil	13408	April 26, 2026
Email	**HeroVired Username**	**Course**
patilkaushal89@gmail.com	Kaushal Patil_13408	Postgraduate Program in Multi Cloud Architecture & DevOps: Batch 16A

GitHub Repository	https://github.com/kaushalpatil205/TravelMemory-AWS-Deployment

1 ASSIGNMENT COMPLIANCE CHECK

The following table verifies that every requirement listed in the assignment has been fully completed and is demonstrable in the live deployment and GitHub repository.

Set up backend on Node.js	✓ Done	Node.js 18 LTS running on TM-Backend-1 and TM-Backend-2
Nginx reverse proxy (port 80→3000)	✓ Done	Configured on both backend EC2 instances
Update .env with DB details & port	✓ Done	PORT=3000 and MONGO_URI set on both backends
Update urls.js for frontend↔backend	✓ Done	Points to https://api.draxil.site
Multiple frontend instances	✓ Done	TM-Frontend-1 and TM-Frontend-2 (t2.micro)
Multiple backend instances	✓ Done	TM-Backend-1 and TM-Backend-2 (t2.micro)
Load balancer — frontend	✓ Done	TM-Frontend-ALB distributes to both frontend instances
Load balancer — backend	✓ Done	TM-Backend-ALB distributes to both backend instances
CNAME record → load balancer endpoint	✓ Done	www.draxil.site → TM-Frontend-ALB DNS
A record → EC2 instance IP	✓ Done	draxil.site → TM-Frontend-1 public IP
Custom domain via Cloudflare	✓ Done	draxil.site active on Cloudflare free plan
HTTPS / SSL	✓ Done	Cloudflare Flexible SSL, Always Use HTTPS enabled
Comprehensive documentation	✓ Done	README.md + 75 screenshots on GitHub
Architecture diagram (draw.io)	✓ Done	PNG + .drawio file in /architecture folder
GitHub repository (public)	✓ Done	github.com/kaushalpatil205/TravelMemory-AWS-Deployment

Result: All 15 assignment requirements are fully satisfied. The application is live, load-balanced, secured with HTTPS, and accessible at https://www.draxil.site.

2 PROJECT OVERVIEW

TravelMemory is a full-stack MERN (MongoDB, Express, React, Node.js) web application that allows users to record and share their travel experiences. The project requirement was to deploy this application on AWS EC2 with high availability, load balancing, and a custom domain through Cloudflare — replicating a real-world production deployment.

Original Source Repository

`https://github.com/UnpredictablePrashant/TravelMemory`

Technology Stack

Technology	Version	Purpose
Node.js 18 LTS	18.x	Backend JavaScript runtime
Express.js	4.x	REST API framework
React.js	18.x	Frontend single-page application
MongoDB Atlas	M0 Free	Cloud-hosted NoSQL database
Nginx	1.18+	Web server and reverse proxy
PM2	Latest	Node.js process manager — keeps app running 24/7
AWS EC2	t2.micro	Virtual servers (free tier eligible)
AWS ALB	-	Application Load Balancer — distributes traffic
Ubuntu Server	22.04 LTS	EC2 operating system
Cloudflare	Free	DNS, SSL termination, CDN, WAF

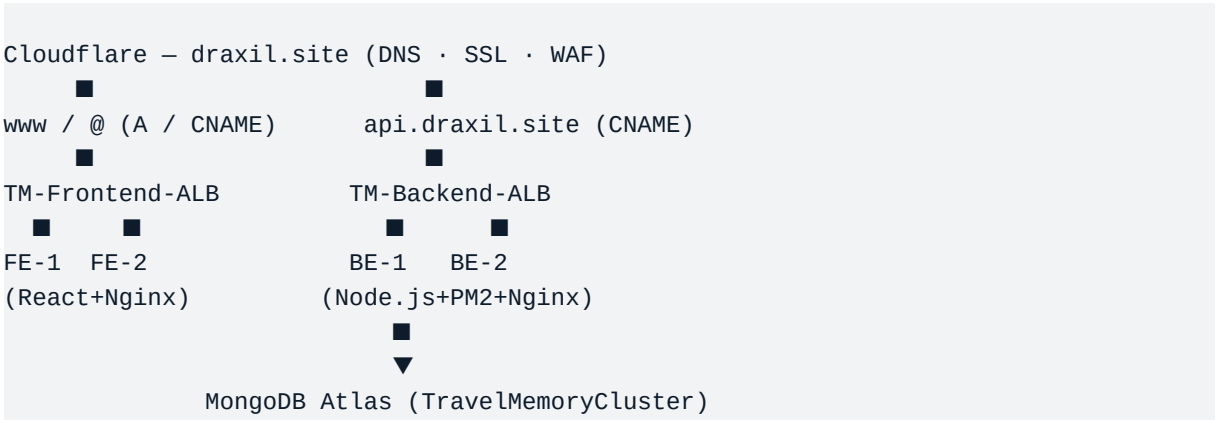
3 ARCHITECTURE OVERVIEW

Traffic Flow (Top to Bottom)

1. User browser sends an HTTPS request to **draxil.site**.
2. **Cloudflare** receives the request, terminates SSL, applies WAF rules, and resolves DNS.
3a. Requests to **www.draxil.site** (CNAME) → **TM-Frontend-ALB** → distributed to **TM-Frontend-1** or **TM-Frontend-2** (React + Nginx).
3b. API calls to **api.draxil.site** (CNAME) → **TM-Backend-ALB** → distributed to **TM-Backend-1** or **TM-Backend-2** (Node.js + PM2 + Nginx).
4. Backend instances connect to **MongoDB Atlas** (TravelMemoryCluster, ap-south-1) via Mongoose ODM over a secure Atlas connection string.

Infrastructure Diagram

```
User Browser (HTTPS)
    ■
    ▼
```



EC2 Instance Summary

Instance	Type	OS	Role	Security Group
TM-Backend-1	t2.micro	Ubuntu 22.04	Node.js API server	TM-Backend-SG
TM-Backend-2	t2.micro	Ubuntu 22.04	Node.js API server	TM-Backend-SG
TM-Frontend-1	t2.micro	Ubuntu 22.04	React static files	TM-Frontend-SG
TM-Frontend-2	t2.micro	Ubuntu 22.04	React static files	TM-Frontend-SG

4 DEPLOYMENT PROCESS SUMMARY

The deployment was executed in 8 sequential phases. Each phase is fully documented with step-by-step commands and screenshots in the GitHub repository README.

Phase 1 AWS Setup	Created Key Pair (travelmemory-key .pem) for SSH access. Created two Security Groups: TM-Backend-SG (ports 22, 80, 3000) and TM-Frontend-SG (ports 22, 80, 443). Selected region: ap-south-1 (Mumbai).	Screenshots 01–04
Phase 2 MongoDB Atlas	Created free M0 cluster (TravelMemoryCluster) on MongoDB Atlas in ap-south-1. Created database user 'tmuser'. Configured network access to allow 0.0.0.0/0. Obtained connection string for use in .env file.	Screenshots 05–09
Phase 3 EC2 Launch	Launched 4 EC2 instances: TM-Backend-1, TM-Backend-2, TM-Frontend-1, TM-Frontend-2. All use Ubuntu 22.04 LTS, t2.micro, travelmemory-key key pair. Noted public IPs of all instances.	Screenshots 10–11
Phase 4 Backend Setup	On both backend instances: installed Node.js 18, Git, Nginx. Cloned repository. Ran npm install. Created .env file with PORT=3000 and MONGO_URI. Tested with node index.js. Started with PM2 (pm2 start index.js --name tm-backend). Configured Nginx reverse proxy (port 80 → 3000). Verified with nginx -t.	Screenshots 12–28
Phase 5 Frontend Setup	On both frontend instances: same system dependencies installed. Cloned repository. Updated src/url.js to point to https://api.draxil.site. Ran npm install and npm run build. Copied build files to /var/www/html. Configured Nginx for React SPA with try_files fallback to index.html.	Screenshots 29–40
Phase 6 Load Balancers	Created TM-Backend-TG target group (HTTP:80, health check /) with both backend instances. Created TM-Frontend-TG with both frontend instances. Created TM-Backend-ALB and TM-Frontend-ALB (Application, Internet-facing, all AZs). Verified both ALBs Active and all 4 targets Healthy.	Screenshots 41–48

Phase 7 Cloudflare DNS	Added draxil.site to Cloudflare. Updated Namecheap nameservers to Cloudflare's. Created DNS records: A record (@→Frontend-1 IP), CNAME www→Frontend ALB, CNAME api→Backend ALB. Set SSL to Flexible (resolved Error 521). Enabled Always Use HTTPS and Automatic HTTPS Rewrites.	Screenshots 49–73
Phase 8 Final Testing	Verified: https://www.draxil.site loads with padlock. https://api.draxil.site responds. HTTP auto-redirects to HTTPS. Both target groups show Healthy. Successfully added a travel memory end-to-end (browser → React → API → MongoDB).	Screenshots 62–71

Key Commands Used

SSH into EC2	`ssh -i ~/Downloads/travelmemory-key.pem ubuntu@`
Install Node.js 18	`curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - && sudo apt install -y nodejs`
Install Nginx	`sudo apt install -y nginx && sudo systemctl enable nginx`
Clone repository	`git clone https://github.com/UnpredictablePrashant/TravelMemory.git`
Install dependencies	`cd TravelMemory/backend && npm install`
Start with PM2	`pm2 start index.js --name tm-backend && pm2 save`
Test Nginx config	`sudo nginx -t && sudo systemctl reload nginx`
Build React app	`cd TravelMemory/frontend && npm run build`
Deploy to Nginx	`sudo cp -r build/* /var/www/html/`

5 CONFIGURATION DETAILS

Nginx — Backend Reverse Proxy

Saved at /etc/nginx/sites-available/tm-backend on both backend EC2 instances. Forwards all port 80 traffic to Node.js running on port 3000.

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Nginx — Frontend React SPA

Saved at /etc/nginx/sites-available/tm-frontend on both frontend EC2 instances. Serves the React build files and handles client-side routing.

```
server {
    listen 80;
    server_name _;
    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, no-transform";
    }
}
```

Environment Variables (.env) — Backend

```
PORT=3000
MONGO_URI=mongodb+srv://tmuser:<password>@travelmemorycluster.xxxxx.mongodb.net/trav
elmemory?retryWrites=true&w=majority
```

Security note: The actual .env file is never committed to GitHub. Only a template (docs/env-sample.txt) is stored in the repository.

Cloudflare DNS Records

Type	Name	Target / Value	Proxy	Purpose
A	@ (root)	TM-Frontend-1 Public IP	Proxied	draxil.site → EC2 directly
CNAME	www	TM-Frontend-ALB-328778329.ap-south-1.elb.amazonaws.com	Proxied	www.draxil.site → Frontend ALB
CNAME	api	TM-Backend-ALB-1289212996.ap-south-1.elb.amazonaws.com	Proxied	api.draxil.site → Backend ALB

6 FINAL TESTING RESULTS

Test	URL / Method	Result
Root domain HTTP	http://draxil.site	✓ React app loads
www with HTTPS	https://www.draxil.site	✓ App loads with padlock
HTTP → HTTPS redirect	http://www.draxil.site	✓ Auto-redirected to HTTPS
API endpoint	https://api.draxil.site	✓ Backend Node.js responds
Backend target health	AWS Console — TM-Backend-TG	✓ Both targets Healthy
Frontend target health	AWS Console — TM-Frontend-TG	✓ Both targets Healthy
Both ALBs active	AWS Console — Load Balancers	✓ Active state confirmed
End-to-end (add memory)	App UI → API → MongoDB	✓ Memory saved and displayed
SSL certificate	Padlock → Certificate details	✓ Cloudflare SSL valid

7 TROUBLESHOOTING ENCOUNTERED

Error 521 — Web Server Down (Cloudflare)

Root cause: Cloudflare SSL was set to "Full" mode. This caused Cloudflare to attempt an HTTPS connection to the EC2 instances on port 443. However, Nginx was only configured to listen on port 80 (HTTP). The connection was refused, resulting in Cloudflare's Error 521.

Fix applied: Changed Cloudflare SSL/TLS mode from "Full" to "Flexible". In Flexible mode, Cloudflare handles HTTPS externally (browser ↔ Cloudflare), and connects to the EC2 instances internally via HTTP on port 80 — which Nginx accepts. The user's browser still receives full HTTPS with a valid padlock.

8 GITHUB REPOSITORY STRUCTURE

https://github.com/kaushalpatil205/TravelMemory-AWS-Deployment

```
TravelMemory-AWS-Deployment/
■■■ README.md                  ← Complete step-by-step deployment documentation
■■■ architecture/
■   ■■■ TravelMemory-AWS-Architecture.png  ← Architecture diagram (PNG)
■   ■■■ TravelMemory-AWS-Architecture.drawio ← Editable draw.io source
■■■ nginx-configs/
■   ■■■ backend-nginx.conf          ← Nginx reverse proxy configuration
■   ■■■ frontend-nginx.conf         ← Nginx React SPA configuration
■■■ docs/
■   ■■■ env-sample.txt             ← .env template (no secrets)
■■■ screenshots/                 ← 75 numbered deployment screenshots
    ■■■ 01-aws-console-region.png
    ■■■ 02-key-pair-created.png
    ■■■ ... (01 through 75)
```

9 SUBMISSION DETAILS

Student Name	Kaushal Bhupendra Patil
HeroVired ID	13408
HeroVired Username	Kaushal Patil_13408
Email	patilkaushal89@gmail.com
Course	Postgraduate Program in Multi Cloud Architecture & DevOps: Batch 16A
Assignment	TravelMemory MERN Stack — AWS EC2 Deployment
Submission Date	April 26, 2026
GitHub Repository	https://github.com/kaushalpatil205/TravelMemory-AWS-Deployment
Live Frontend	https://www.draxil.site
Live API	https://api.draxil.site
Root Domain	https://draxil.site

Repository visibility: The GitHub repository github.com/kaushalpatil205/TravelMemory-AWS-Deployment is set to **Public** and is accessible without login. The README contains full documentation with all 75 screenshots embedded.

IMPORTANT NOTE

All EC2 instances, Load Balancers, Target Groups, and related AWS infrastructure have been terminated and deleted to avoid ongoing AWS charges after the assignment submission. The live URLs may no longer be accessible. All screenshots documenting the complete working deployment are uploaded to the GitHub repository as proof of work and successful completion of the assignment.

Submitted by **Kaushal Bhupendra Patil** | HeroVired ID: **13408** | **April 26, 2026**